

Bridging Query Completion and Item Recommendation in a Multi-Round Guess-and-Update System

Jiarui Jin^{1,*}, Xianyu Chen^{1,*}, Weinan Zhang¹, Yong Yu¹, Jun Wang²

¹Shanghai Jiao Tong University, ²University College London

{jinjiarui97,xianyujun,wnzhang,yyu}@sjtu.edu.cn,jun.wang@cs.ucl.ac.uk

ABSTRACT

Query completion systems (a.k.a., query recommender systems) endeavor to forecast the queries that users are inclined to search, whereas item (a.k.a., document) recommender systems focus on predicting the items that users are likely to prefer. The main idea put forth in this paper is that both query completion systems and item recommender systems share the objective of capturing user interests, differing primarily in the level of granularity at which they operate. Query recommender systems primarily operate at a coarse level by predicting user search queries, while item recommender systems operate at a finer level by predicting user preferences for specific items. In light of this, we propose a novel approach that aims to integrate and leverage the strengths of both systems. This unified system, referred to as Multi-round Auto Guess-and-Update System (MAGUS), introduces a recursive framework that could be applied to *any* recommender system to enable the recommendation of both queries and items at each round. Concretely, we begin by representing queries and items as combinations of specific categorical words. We then construct a relational graph that captures the connections and dependence among these individual words and combinations of words. For each session (i.e., user), we assign the estimated relevance scores given by the recommender system to those words representing items. By the graph, we can efficiently propagate the scores on the graph. Next, we guess user preferences by forming meaningful combinations of words and presenting them as potential queries or items. In a multiple-round setting, MAGUS is supposed to receive user feedback at each round. We design a new label propagation algorithm to efficiently simulate the influence of feedback throughout the graph of interconnected nodes, thereby updating their prediction scores. Experimental results on three benchmark datasets demonstrate that integrating query recommendations with item recommendations by MAGUS yields benefits for users in terms of finding their satisfying items more efficiently. One typical use case of MAGUS is the exploratory search, where users do not have a clear picture of the items they demand; and our framework allows users to browse and compare items before re-formulating or selecting new queries.

KEYWORDS

Query Completion, Item Recommendation, Relational Graph, Feature Propagation, Label Propagation

1 INTRODUCTION

The advancements in search engine technology have given rise to two primary types of recommendation services. The first type is query completion (a.k.a., query recommendation) [5], which focuses on alleviating users from the burden of manually inputting entire queries by providing them with relevant and predictive suggestions. The second type is item (a.k.a., document) recommendation [25], whose objective is to suggest preferred or favored items to users.

We notice that the existing utilization of query completion and item recommendation systems in numerous e-commerce applications (e.g., Amazon and Alibaba) has largely been independent of each other. We posit that there should exist untapped potential for mutual benefit between these systems. Despite their operational disparities, both systems fundamentally aim to capture user interests. Their primary distinction lies in the level of granularity at which they operate: query completion systems focus on predicting user queries at a coarse level, while item recommendation systems target specific items at a finer level.

Given this shared objective of understanding user interests, we argue that integrating these systems can benefit both of them. The user's interaction with the recommended items can spark more refined query suggestions, while the user's response to the suggestions can further inform the item recommendations. One typical use case is the exploratory search, where users often have a vague or unclear idea of the specific items they are seeking. Taking Figure 1 as an example, a user intends to purchase milk but lacks specific criteria such as the preferred brand or price range to give a well-formed query. In such a scenario, a good choice is to provide a set of recommended milk items together with suggested queries, allowing users to engage in browsing and comparing various items before refining their search queries. Achieving this requires a system to jointly consider query completion and item recommendation.

In this paper, we propose a novel Multi-round Auto Guess-and-Update System (MAGUS), a recursive framework, which can be applied to *any* recommender system to enable it to recommend both queries and items at each round, and identify the items that meet the user's needs for each session. When designing such a system, we are, at least, required to handle the following challenges:

- [C1] How to draw connections between recommending queries and items? Taking Figure 1 as an example, an item recommender

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Pre-print, Under review., 2023

© 2023 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

*Equal contributions: Jiarui and Xianyu.

[†]Work done during Jiarui's visit at University College London.

system (as shown in (a)) only needs to compare among milk items while a query completion system (as shown in (b)) seeks relevant queries prefixed by the term milk. Therefore, the core challenge of recommending both queries and items is to develop a unified measurement of queries and items.

- [C2] How to capture the dependence among queries? Different from items, there exists strong dependence among queries. Roughly speaking, for each query-query pair, there are three possible scenarios: (i) Mutual improvement: Selecting one query increases the likelihood of selecting the other query in the following round (e.g., selecting Milk would raise the probability of selecting Whole Milk in the next round). (ii) Mutual inhibition: Selecting one query decreases the probability of selecting the other query in the following round (e.g., if a user selects Milk, then it is unlikely for her to select Beef, because milk and beef belong to two distinct categories). (iii) Mutual Independence: The selection of one query has minimal or negligible effects on the user’s decision regarding the other query (e.g., selecting Milk does not significantly influence the user’s preference for On Sale as a query).
- [C3] How to efficiently make use of user feedback at each round subjecting to strict online latency requirements? One of the main advantages of multiple-round recommendations is that it allows gradually reaching user interests by recursively guessing and updating. This recursive framework requires efficiently exploiting user feedback while adhering to strict online latency requirements can be challenging.

To tackle the above challenges, we note that both queries and items serve as representations of user interests, although they operate at different levels of granularity. Queries capture user needs at a higher, more abstract level, while items provide specific, detailed products satisfying user demands. In this regard, both queries and items are combinations of specific categorical words. As exemplified in Figure 1, Item A in (a) could be expressed as the combination of words Brand A, Milk, Semi Skimmed, 2L, and \$3; and Query B could be organized as the combination of words Whole and Milk. Then, queries and items can be connected through the shared use of words, addressing [C1]. To capture the dependence among the words, we construct a relational graph, where the nodes correspond to individual words or combinations of words, and the edges represent three types of dependence introduced in [C2]. We illustrate the graph in Figure 2(a).

For each session, we are given a recommender system to initialize the predicted score for each node representing in the graph, e.g., Item A, Item B, Item C in Figure 2(a). We then propagate the scores to related words through the graph, as exemplified in Figure 2(b). At each round, MAGUS sorts all the nodes and generates a list of high-scored nodes. These nodes can be either queries or items. In a multiple-round setting, MAGUS should receive feedback from users at each round. To efficiently leverage user feedback while adhering to strict online latency requirements (i.e., [C3]), MAGUS incorporates a label propagation algorithm to simulate the influence of user feedback on the words involved as shown in Figure 2(c). We also provide a feature propagation based approach to learn the weights for the edges, allowing us to operate weighted propagation in Figure 2(b) and (c).

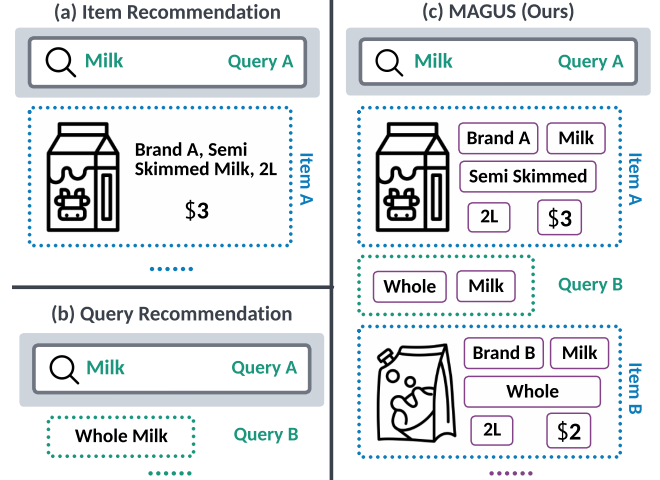


Figure 1: An illustrated example of MAGUS recommending both queries and items, as shown in (c), which combines the capabilities of the item recommender system, as shown in (a), and the query completion system, as shown in (b).

We note that our objective for each session is to find an item satisfying the user’s demands. Therefore, at the beginning of each session, the predicted scores are initialized from the nodes representing items to other nodes. These scores provided by any recommender system, serve as a starting point. As the session progresses, the update occurs from the nodes receiving user responses to the nodes representing items. By initializing the scores from an offline recommender system and updating them based on online multiple-round user responses, MAGUS can dynamically adjust the scores of nodes matching the user needs throughout the session.

We establish a novel multiple-round recommendation simulator. The MAGUS agent operates in a recursive manner, suggesting queries and items at each session, while the user agent provides corresponding responses. This iterative process continues until an item that satisfies the user’s needs is found. Our experimental results on three real-world datasets demonstrate that MAGUS, when incorporated with nine different recommender systems, can consistently improve the recommendation performance in terms of finding satisfying items for the given user.

2 PRELIMINARIES

2.1 Problem Formulation

We begin by introducing the notations used in this paper. $\mathcal{U}$ denotes the set of users, $\mathcal{V}$ denotes the set of items, and $\mathcal{Q}$ denotes the set of queries. For convenience, we use $\psi_{\text{RE}}(\cdot)$, a mapping function $\psi_{\text{RE}} : \mathcal{U} \times \mathcal{V} \rightarrow \mathbb{R}$, to denote any given recommender system that assigns a predicted relevance score to each item. We use $\psi_{\text{RE}}(v)^*$ to denote the score of each item $v \in \mathcal{V}$. Our objective is to build a joint mapping function $\psi_{\text{MAGUS}} : \mathcal{U} \times \mathcal{A} \rightarrow \mathbb{R}$, where $\mathcal{A} = \mathcal{V} \cup \mathcal{Q}$ is the action space. In other words, our MAGUS, i.e., $\psi_{\text{MAGUS}}(\cdot)$, can recommend both queries and items.

Notice that both queries and items represent user interests, albeit at different levels of granularity. Therefore, MAGUS can leverage

^{*}We use omit u in notations (i.e., using $\psi_{\text{RE}}(v)$ instead of $\psi_{\text{RE}}(u, v)$) for simplicity.

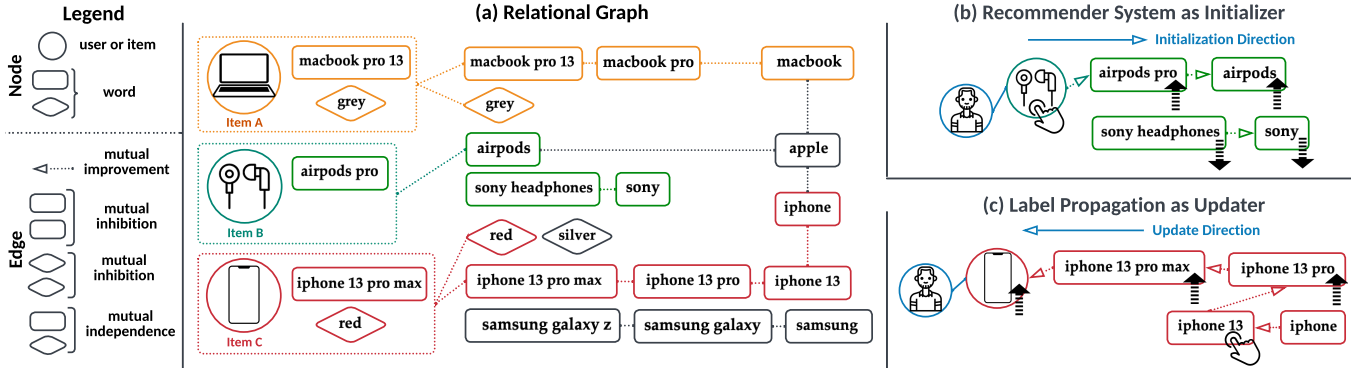


Figure 2: An illustrated example of our relational graph organizing individual words and some combinations of words (as shown in (a)), where we do not explicitly draw edges of mutual inhibition and mutual independence for clarity. In each session, MAGUS can incorporate with any recommender system to initialize the scores of the nodes where the propagation direction is from the nodes representing items to the other nodes (as shown in (b)); when a user provides her preference on a recommendation, MAGUS updates the subsequence nodes by a label propagation algorithm whose propagation direction is from the recommended node to the nodes representing items (as shown in (c)).

this distinction when determining the appropriate recommendation to make based on the understanding level of user demands. Figure 1(c) deploys an illustrated example, where Item A, Item B are fine-level representations while Query A is a coarse-level representation.

In this paper, we investigate a multiple-round recommendation setting in each session. For clarity, we pose the description in the context of the top-1 recommendation, and discuss the extension to the top-N recommendation in Section 4.2.

Definition 2.1 (MULTIPLE-ROUND RECOMMENDATIONS FOR BOTH QUERIES AND ITEMS). Given a tuple $(\mathcal{U}, \mathcal{V}, \mathcal{Q}, K_{\text{MAX}})$, in each session (with user $u \in \mathcal{U}$), the goal of $\psi_{\text{MAGUS}}(\cdot)$ is to recommend an item satisfying u 's needs within K_{MAX} interactions. The action space of $\psi_{\text{MAGUS}}(\cdot)$ covers both item space and query space, namely, $\mathcal{A} = \mathcal{V} \cup \mathcal{Q}$. For each k -th round (where $k = 1, \dots, K_{\text{MAX}}$), $\psi_{\text{MAGUS}}(\cdot)$ recommends either an item or a query, and the user should provide either positive or negative feedback. Formally, let $\mathcal{V}_{\text{TARGET}}$ denote the set of items that can satisfy the user's need, and let $a_k \in \mathcal{A}$ denote the recommendation made by $\psi_{\text{MAGUS}}(\cdot)$ at k -th round. Our objective can be formulated as:

$$\min K, \text{ s.t., } a_K \in \mathcal{V}_{\text{TARGET}}, \quad (1)$$

where $K = 1, \dots, K_{\text{MAX}}$. In other words, an oracle $\psi_{\text{MAGUS}}(\cdot)$ is supposed to find an item in $\mathcal{V}_{\text{TARGET}}$ with a minimal number of rounds.

Remark. Note that query recommendations are significantly different from attribute recommendations investigated in conversational recommender systems [3, 19, 28, 37]. The major difference is that there is a strong dependence among queries (e.g., apple and macbook, macbook and macbook pro), while the same level of dependence does not exist among attributes (e.g., brand and color). This distinction arises due to the nature of the information being represented: users typically refine their search or express their intent by adding or modifying queries, while attributes are used to represent different aspects or characteristics of an item.

2.2 Connections to Previous Work

Multiple-round Item Recommender Systems. Conversational recommender systems [11, 17] allow users to express their opinions about recommended items at each round, to guide the system in providing further recommendations. Along this line, one popular direction is to a conversational recommender system [19, 20, 28, 37], which combines the conversational models and the recommendation models from a systematic perspective. For example, compared to previous literature [8, 9, 35], a paper [38] extends a single-round system to the multiple-round settings, and papers [19, 21] develop a reinforcement learning framework allowing optimizations over multiple-round interactions. However, reinforcement learning based approaches are innately suffering from insufficient usage of labeled data and high complexity costs of deployment. In contrast, MAGUS can be seamlessly adopted to any recommendation method, and enables it to recommend both queries and items.

Query Recommender Systems. Query auto-completion systems [5] are designed to return a list of completed queries that match the inputs from users (a.k.a., prefix queries). This field is also highly related to query recommendation (a.k.a., query suggestion or query expansion) [29]. Previous literature producing completed queries can be roughly categorized into retrieval-based or generation-based methods. Retrieval-based direction leverages the expressive power of deep neural networks and develops dense retrieval approaches [1, 7, 30]. These approaches are inherently restricted by what exists in the logs and thus suffer from the lack of appropriate candidates for tail queries [10]. To overcome this limitation, generation-based strategies have been pursued regarding the query suggestion as a natural language generation problem [6]. Our paper is along the line of retrieval-based approaches; and different from previous work, our retrieval pool includes both queries and items. By combining item and query recommendations, MAGUS can leverage the interactions of both query and item recommendation logs to handle tail queries.

Graph-based Recommender Systems. Graphs, especially heterogeneous graphs [31] (a.k.a., heterogeneous information networks),

have been widely used to characterize complex objects and relations. Previous investigations of using graphs in information retrieval mainly develop heterogeneous graph versions of graph neural networks such as [26, 33, 34, 36], or incorporate graph neural networks to collaborative filtering techniques such as [32]. Almost all of these methods can be unified under a general framework [12] known as message passing or feature propagation. In contrast to computationally expensive feature propagation on a user-item graph, our paper first organizes the connections between queries and items by a word graph and then proposes to use a label propagation algorithm to leverage the predictions of any recommender system. One of our major advantages is the efficiency in an online setting, where we can make use of user feedback in real-time while meeting the strict latency requirements of online systems.

3 THE MAGUS SYSTEM

3.1 Bridging Queries and Items via Word Graph

A core challenge of bridging query recommendations and item recommendations is to establish a connection between queries and items. In this paper, we propose to use “words” as the bridge.

Definition 3.1 (WORDS AND RELATIONAL GRAPH). Given a tuple $(\mathcal{V}, \mathcal{Q})$, we construct $\mathcal{W}$, the set of categorical words, to connect $\mathcal{V}$ and $\mathcal{Q}$. Concretely, we first extract the words from the categorical attributes (a.k.a. features) of items in $\mathcal{V}$, e.g., extracting Brand A, Milk, Semi Skimmed, 2L, \$3 from Item A. These extracted words constitute $\mathcal{W}$. We then decompose queries in $\mathcal{Q}$ by matching them with the words in $\mathcal{W}$, e.g., Query B is decomposed as the combination of Whole and Milk. We build a relation graph, denoted as $\mathcal{G} = (\mathcal{E}, \mathcal{R})$, where $\mathcal{E}$ is the set of nodes and $\mathcal{R}$ is the set of edges. $\mathcal{E}$ consists of individual words from the vocabulary $\mathcal{W}$, as well as combinations of words that form queries in the query set $\mathcal{Q}$ and items in the item set $\mathcal{V}$. For example, as illustrated in Figure 2(a), node (macbook pro 13 grey) represents Item A, and node (macbook pro 13) and node (macbook pro) are the combinations of words used to organize Item A, and node (grey) and node (macbook, apple) are individual nodes. $\mathcal{R}$ contains three types of relations between nodes, i.e., mutual improvement (denoted as $\mathcal{R}_+$), mutual inhibition (denoted as $\mathcal{R}_-$), and mutual independence (denoted as $\mathcal{R}_\perp$). In other words, $\mathcal{R} = \mathcal{R}_+ \cup \mathcal{R}_- \cup \mathcal{R}_\perp$. $\mathcal{R}_+$ is established between pairs of nodes where one node is a part of the other node, e.g., node (macbook) is a part of node (macbook pro). $\mathcal{R}_-$ exists in pairs of nodes that are used to represent different items, e.g., node (macbook pro) and node (iphone pro) are used to organize Item A and Item C respectively. $\mathcal{R}_\perp$ connects all other pairs of nodes, i.e., pairs of nodes used to represent the same item and have no overlapping part, e.g., node (macbook pro) and node (gray) are used to form Item A and do not have any overlapping.

From Definition 3.1, it is evident that all the queries and items can be mapped to their corresponding nodes in the graph, i.e., $\mathcal{Q} \subset \mathcal{E}$ and $\mathcal{V} \subset \mathcal{E}$. We note that the process of constructing the relational graph as described above is a general pipeline that should be tailored and specified for specific datasets[†].

[†]Take Figure 2(a) as an example. In $\mathcal{R}_+$, node (apple) is part of node (apple macbook) where the brand apple is often omitted for simplicity, and in $\mathcal{R}_-$, we only draw the

3.2 Recommender System as Initializer

In each session (with user $u \in \mathcal{U}$), given a pre-(trained) recommender system $\psi_{\text{RE}}(\cdot)$, we can assign an estimated relevance score for each item $v \in \mathcal{V}$, denoted as $\psi_{\text{RE}}(v)$. According to Definition 3.1, each item in $\mathcal{V}$ corresponds to a node in $\mathcal{E}$. For convenience, let $\mathcal{E}_{\text{ITEM}}$ denote the set of nodes representing items. Let $\hat{y}_v$ represent the predicted relevance score for each node $v \in \mathcal{E}$.

We first assign the predicted scores by the recommender system to the nodes in $\mathcal{E}_{\text{ITEM}}$, and initialize the other nodes as 0. Formally, for each node $v \in \mathcal{E}$, we have:

$$\hat{y}_v = \begin{cases} \psi_{\text{RE}}(v), & v \in \mathcal{E}_{\text{ITEM}}, \\ 0, & v \in \mathcal{E} - \mathcal{E}_{\text{ITEM}}. \end{cases} \quad (2)$$

For convenience, we scale all the scores within the range of 0 to 1, namely $\psi_{\text{RE}}(v) \in [0, 1]$ holds for every $v \in \mathcal{E}_{\text{ITEM}}$.

We then propagate these scores to all the other nodes in $\mathcal{E}$. For this purpose, for each edge in $\mathcal{R}$, we assign a learnable weight. For simplicity, we use $w_{v'v''}$ to denote the weight of the edge connecting node v' and node v'' (and a detailed description of computing the weights is later introduced in Section 3.4). As shown in Figure 2(b), the propagation direction starts from the nodes in $\mathcal{E}_{\text{ITEM}}$ to the other nodes. The update function of the nodes in $\mathcal{E} - \mathcal{E}_{\text{ITEM}}$ during the propagation can be written as:

$$\hat{y}_{v'} \leftarrow \begin{cases} \hat{y}_{v'} + w_{v'v''} \cdot \hat{y}_{v''}, & \langle v', v'' \rangle \in \vec{\mathcal{R}}_+, \\ \hat{y}_{v'} - w_{v'v''} \cdot \hat{y}_{v''}, & \langle v', v'' \rangle \in \vec{\mathcal{R}}_-, \\ \hat{y}_{v'}, & \langle v', v'' \rangle \in \mathcal{R}_\perp \cup \overleftarrow{\mathcal{R}}_+ \cup \overleftarrow{\mathcal{R}}_-, \end{cases} \quad (3)$$

where $v' \in \mathcal{E} - \mathcal{E}_{\text{ITEM}}$ and $v'' \in \mathcal{E}$, and $\vec{\mathcal{R}}_+$ and $\vec{\mathcal{R}}_-$ are introduced to denote the update directions on the edges in $\mathcal{R}_+$ and $\mathcal{R}_-$ respectively. Here, $\mathcal{R}_+ = \vec{\mathcal{R}}_+ \cup \overleftarrow{\mathcal{R}}_+$ and $\mathcal{R}_- = \vec{\mathcal{R}}_- \cup \overleftarrow{\mathcal{R}}_-$.

Then, at each round, we generate the recommendations by sorting all the nodes and selecting the one, representing either a query or an item, with the highest predicted scores. Formally, we have:

$$a_{\text{MAGUS}} = \arg \max_{a \in \mathcal{A}} \psi_{\text{MAGUS}} = \arg \max_{v \in \mathcal{A}} \hat{y}_v, \quad (4)$$

where a_{MAGUS} denotes the query or item to recommend by MAGUS.

3.3 Label Propagation as Updater

To efficiently exploit the real-time feedback from users, we propose a label propagation based algorithm to simulate the impacts of user feedback on the other nodes. For each recommendation a_{MAGUS} , we update the corresponding node, denoted as v , based on the feedback received. The update function can be written as:

$$\hat{y}_v \leftarrow \begin{cases} 1, & v \in \mathcal{E}_{\text{POS}}, \\ 0, & v \in \mathcal{E}_{\text{NEG}}, \end{cases} \quad (5)$$

where $v \in \mathcal{E}_{\text{POS}}$ denotes the event where u gives positive feedback (e.g., clicks) to v , and $v \in \mathcal{E}_{\text{NEG}}$ denotes the event where u provides negative feedback (e.g., observations without clicks) to v .

edges between pairs of category nodes (e.g., node (macbook pro) and node (iphone 13 pro)) and pairs of color nodes (e.g., node (gray) and node (red)).

We then propagate the score of v to the other nodes, i.e., $v' \in \mathcal{E} \setminus \{v\}$. Therefore, our update function can be formulated as:

$$\hat{y}_{v'} \leftarrow \begin{cases} \text{MIN}(1, \hat{y}_{v'} + w_{v'v''} \cdot \hat{y}_{v''}), & \langle v', v'' \rangle \in \vec{\mathcal{R}}_+, \\ \text{MAX}(0, \hat{y}_{v'} - w_{v'v''} \cdot \hat{y}_{v''}), & \langle v', v'' \rangle \in \vec{\mathcal{R}}_-, \\ \hat{y}_{v'}, & \langle v', v'' \rangle \in \mathcal{R}_\perp \cup \vec{\mathcal{R}}_+ \cup \vec{\mathcal{R}}_-, \end{cases} \quad (6)$$

where $\text{MAX}(\cdot)$ and $\text{MIN}(\cdot)$ are the maximum function and the minimum function. We note that in contrast to Eq. (3), which propagates information in the direction of $\vec{\mathcal{R}}_+ \cup \vec{\mathcal{R}}_-$, the propagation direction in Eq. (6) is reversed, i.e., $\vec{\mathcal{R}}_+ \cup \vec{\mathcal{R}}_-$.

After all the nodes in $\mathcal{E}$ are updated at each round, we then apply Eq. (4) to generate a_{MAGUS} at the next round.

3.4 Feature Propagation as Weight Trainer

In this subsection, we describe how to calculate the weight of each edge. For a given recommender system, i.e., $\psi_{\text{RE}}(\cdot)$, if we have no access to the pre-trained representation vectors for all the items and users, then we define can the weights of the edges in $\mathcal{R}$ as:

$$w_{vv'} = 1 \text{ if } \langle v, v' \rangle \in \mathcal{R}_+ \cup \mathcal{R}_-, \text{ and } w_{vv'} = 0 \text{ otherwise.} \quad (7)$$

Let $\mathbf{e}_v^{\text{RE}}$ denote the representation vector of each item $v \in \mathcal{E}_{\text{ITEM}}$, and let $\mathbf{e}_u^{\text{RE}}$ denote the representation vector of each user $u \in \mathcal{U}$. For each session (with each user $u \in \mathcal{U}$), we use $\mathcal{H}_+$ and $\mathcal{H}_-$ to respectively denote the set of items previously received positive feedback from u and the set of items previously received negative feedback from u .

Suppose that we have access to all the $\mathbf{e}_v^{\text{RE}}$ s, and $\mathcal{H}_+$ s and $\mathcal{H}_-$ s for all the users. Then, we design a feature propagation algorithm to learn the weights. Formally, we first initialize the representation vector of each node $v \in \mathcal{E}$ as:

$$\mathbf{e}_v = \mathbf{e}_v^{\text{RE}} \text{ if } v \in \mathcal{E}_{\text{ITEM}}, \text{ and } \mathbf{e}_v = \mathbf{e}_v^{\text{INIT}} \text{ otherwise.} \quad (8)$$

where $\mathbf{e}_v^{\text{INIT}}$ represents the initialized embedding vector for node v following a given initializer such as [13].

We then propagate the feature based on the graph $\mathcal{G}$. The update function of each node $v \in \mathcal{E}$ can be formulated as:[‡]

$$\mathbf{e}_v \leftarrow \text{ReLU}\left(\mathbf{m}_{v \leftarrow v} + \sum_{v' \in \mathcal{N}_+(v)} \frac{\mathbf{m}_{v \leftarrow v'}}{|\mathcal{N}_+(v)|} - \sum_{v'' \in \mathcal{N}_-(v)} \frac{\mathbf{m}_{v \leftarrow v''}}{|\mathcal{N}_-(v)|}\right), \quad (9)$$

where $\text{ReLU}(\cdot)$ is an activation function, $\mathcal{N}_+(v)$ denotes the set of neighbor nodes connected to v through $\mathcal{R}_+$, and $\mathcal{N}_-(v)$ denotes the set of neighbor nodes connected to v through $\mathcal{R}_-$. $\mathbf{m}_{v' \leftarrow v}$ denotes the messages being propagated from node v to node v' . We compute the messages as follows.

$$\mathbf{m}_{v \leftarrow v'} = \begin{cases} \mathbf{W}_1 \cdot \mathbf{e}_v, & v = v', \\ \mathbf{W}_1 \cdot \mathbf{e}_{v'} + \mathbf{W}_2 \cdot (\mathbf{e}_v \odot \mathbf{e}_{v'}), & v \neq v', \end{cases} \quad (10)$$

where $\mathbf{W} \in \mathbb{R}^{d \times d}$, d is the dimension of the embedding vector of nodes, i.e., $\mathbf{e}_v, v \in \mathcal{E}$, and $\odot$ denotes the element-wise product.

To supervise the feature propagation, we leverage the historical records $\mathcal{H}_+$ and $\mathcal{H}_-$ of each user u to establish a log-loss as:

$$L = - \sum_{v \in \mathcal{H}_+ \cup \mathcal{H}_-} \left(y_v \log \hat{y}_v + (1 - y_v) \log(1 - \hat{y}_v) \right), \quad (11)$$

[‡]We do not explicitly write the propagation layers in Eqs. (9) and (10) to make it simple and easy to read. In our experiment, to save the computation cost, we set the number of propagation layers as 1.

Algorithm 1 The MAGUS System

INPUT: any recommender system $\psi_{\text{RE}}(\cdot)$, maximum number of rounds K_{MAX} ; optional: embedding vectors of all the items and users $\{\mathbf{e}_v^{\text{RE}} | v \in \mathcal{E}_{\text{ITEM}}\}$ and $\{\mathbf{e}_u^{\text{RE}} | u \in \mathcal{U}\}$, positive and negative browsed items $\mathcal{H}_+$ and $\mathcal{H}_-$.

OUTPUT: recommended query or item a_{MAGUS} at each round.

- 1: Construct and relational graph $\mathcal{G}$ according to *Definition 3.1*.
 - 2: Compute the weights of the edges in $\mathcal{R}$ using Eq. (7) or (12).
 - 3: **for** each session (i.e., the given user) **do**
 - 4: Initialize $k = 0$.
 - 5: Initialize the scores of all the nodes using Eqs. (2) and (3).
 - 6: **repeat**
 - 7: Compute a_{MAGUS} using Eq. (4).
 - 8: Recommend a_{MAGUS} and receive the response.
 - 9: Update the scores of the nodes using Eqs. (5) and (6).
 - 10: Go to next round: $k \leftarrow k + 1$.
 - 11: **until** $a_{\text{MAGUS}} \in \mathcal{V}_{\text{TARGET}}$ or $k > K_{\text{MAX}}$.
 - 12: Collect session data into $\mathcal{H}_+$ and $\mathcal{H}_-$.
 - 13: **end for**
 - 14: Update $\psi_{\text{RE}}(\cdot)$ using data in $\mathcal{H}_+$ and $\mathcal{H}_-$.
-

where $\hat{y}_v$ is our predicted score relevance for node v regarding user u , calculated by $\hat{y}_v = (\mathbf{e}_u^{\text{RE}})^\top \cdot \mathbf{e}_v$, and y_v is the ground-truth label of node v . $y_v = 1$ if $v \in \mathcal{H}_+$, and $y_v = 0$ if $v \in \mathcal{H}_-$.

Following the above feature propagation process, we can get an updated representation vector for each node. Then, we define the weight of each edge in $\mathcal{R}$ by the similarity between its connected node pair. Formally, we have:

$$w_{vv'} = \mathbf{e}_v^\top \cdot \mathbf{e}_{v'} \text{ if } \langle v, v' \rangle \in \mathcal{R}_+ \cup \mathcal{R}_-, \text{ and } w_{vv'} = 0 \text{ otherwise.} \quad (12)$$

We note that the supervised learning process described above can be deployed in an offline manner. In other words, our relational graph $\mathcal{G}$ can be pre-constructed following *Definition 3.1*, where the edge weights can be computed by either Eq. (7) or (12) and be saved in $\mathcal{G}$. $\mathcal{G}$ can be stored and only need to update when our recommender system $\psi_{\text{RE}}(\cdot)$ is updated.

4 SYSTEM ANALYSIS

4.1 Overall Algorithm

We summarize the MAGUS system in Algorithm 1, where lines 1, 2 and 14 are the offline training part, and lines from 4 to 12 are the online update part. As discussed above, the offline training part can be pre-computed and stored.

The main components of MAGUS system are the feature propagation part in offline training and the label propagation part in online inference. As stated in [32], during each propagation, the matrix multiplication has computational complexity $O(|L_+| \cdot d^2)$. $|L_+|$ is the number of nonzero entities in the Laplacian matrix L . $L = D^{-1/2} A D^{-1/2}$ where A is the adjacency matrix and D is the diagonal degree matrix of graph $\mathcal{G}$. For the feature propagation, d is the dimension of the embedding vector of nodes; while for the label propagation, d is the dimension of the label vector of nodes.

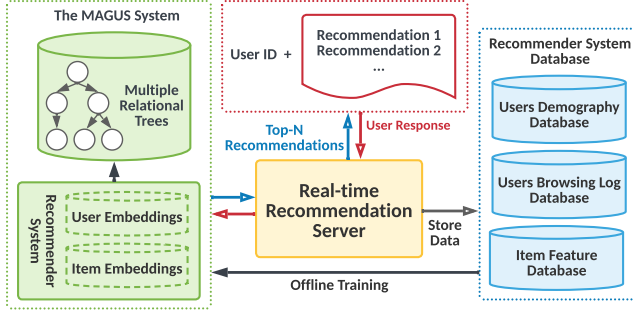


Figure 3: Online deployment of the MAGUS system can be integrated into existing real-time recommendation servers, where solid arrows represent the offline training phase and hollow arrows represent the online update phase. Most components except multiple relational trees (storing the relational graph) already exist in current recommender systems.

4.2 Extensions to Top-N Recommendations

To extend MAGUS from top-1 recommendations to top-N recommendations, we modify Algorithm 1 as follows.

First, instead of selecting only one a_{MAGUS} in line 7, we recursively select a list of B a_{MAGUS} s as recommendations. These a_{MAGUS} s are representing either queries or items, and they are selected based on the top- B predicted scores from $\psi_{\text{RE}}(\cdot)$.

Second, the response from a user and the corresponding updates are different from lines 8 and 9. Different from binary user response (i.e., either positive or negative one) to a recommendation, the response to top-N recommendations can be categorized into two following types. (i) If the user clicks on one of the B a_{MAGUS} s, denoted as a_{MAGUS}^* , we set the score of the node representing a_{MAGUS}^* as 1, and the scores of the nodes representing the remaining $B - 1$ a_{MAGUS} s as 0. This can be done by applying Eq. (5) accordingly. (ii) If the user does not click on any of the B a_{MAGUS} s, we assign the scores of the nodes representing B a_{MAGUS} s as 0. After that, we apply Eq. (6) to propagate the updated scores to the other nodes in the graph.

4.3 Deployment Feasibility

Here, we discuss the feasibility of the industrial deployment of our MAGUS system. Integrating the MAGUS system into the existing recommendation platform would not impose a heavy workload, because the main change brought by MAGUS is in organizing the relational graph and updating the graph by user responses. The majority of the prediction model pipeline remains the same.

To accelerate the computation costs associated with using the graph multiple times, we propose simplifying the graph by creating multiple relational trees, as depicted in Figure 3. Each tree starts from a root node that represents an item and extends to nodes representing individual words. Taking Figure 2(a) as an example, one possible tree grows from the node presenting Item A to the nodes representing individual words (apple) and (grey). This simplification enables parallelization and localized computations, which can significantly improve the efficiency of operations.

5 EXPERIMENTS

5.1 Datasets and Data Preprocessing

Dataset Description. We conduct extensive experiments on 3 industrial real-world e-commerce datasets: **Amazon**[§] is a dataset collected by Amazon from May 1996 to July 2014. There are 1,114,563 reviews of 133,960 users and 431,827 items with an average sequence length of 33 and 6 feature fields. **Alipay**[¶] is a dataset collected by Alipay, from July 2015 to November 2015. There are 35,179,371 interactions of 498,308 users and 2,200,191 items with an average sequence length of 70 and 6 feature fields. **Tmall**^{||} is a dataset collected by Tmall from May 2015 to November 2015. It consists of 54,925,331 interactions of 424,170 users and 1,090,390 items with an average sequence length of 129 and 9 feature fields.

For each dataset, we can directly obtain the user-item interaction histories (i.e., $\mathcal{H} = \mathcal{H}_+ \cup \mathcal{H}_-$) and the categorical features of users and items (i.e., $\mathcal{U}, \mathcal{V}$). According to Definition 2.1, we still need to construct the query set $\mathcal{Q}$. Take Amazon dataset as an instance. We first build the word set $\mathcal{W}$ from the categorical features of items in $\mathcal{V}$, e.g., one item has brand Coxlures and belongs to categories are Sports and Dance, and then we can build words coxlures, sport s, and dance. By enumerating all the items in $\mathcal{V}$, we can obtain $\mathcal{W}$. In this case, these individual words can compose the combinations (coxlures, sport and dance) representing Item A and (sport and dance) whose prefixes include individual words sport and dance. All the combinations of words and individual words are used as nodes to construct the relational graph (i.e., $\mathcal{E}$). These nodes, except for those representing items, can be considered as queries (i.e., $\mathcal{Q}$).

Data Preprocessing. For each dataset, we split the dataset into training/validation/test sets at 6:2:2 by timestep. We remove sequences that either have a length smaller than 30 or do not contain any item that received positive feedback. We then randomly select 30 items to form a session, and we impose a constraint during the random selection process to guarantee that at least one item receiving positive feedback is selected. We assign all the items receiving positive feedback as the target items in each session.

Simulator Design. We establish a new recommendation simulator, consisting of the user agent and the recommender agent. The recommender agent can either select an item or a query at each round. The user agent’s response depends on whether the recommendation a_{MAGUS} corresponds to a target item or a query that is part of a target item. If it does, the user agent provides a positive response. Otherwise, it gives a negative response. Each session ends when the recommender agent recommends a target item to the user agent.

All the preprocessed datasets (together with the simulator) and the codes will be released at publication time.

5.2 Baselines in Multiple-Round Setting

Baseline Description. The main insight of your paper is the integration of query recommendations and item recommendations. Here, we introduce 11 recommendation approaches. MPS (Most Popular Suggestion) [10, 27] scores each query by normalizing the

[§]<https://jmcauley.ucsd.edu/data/amazon/>

[¶]<https://tianchi.aliyun.com/dataset/dataDetail?dataId=53>

^{||}<https://tianchi.aliyun.com/dataset/dataDetail?dataId=42>

co-occurrence to categories (e.g., brands, colors) in the browsing item logs. **Hybrid** (Hybrid Suggestion) [2] uses a linear combination between the popularity and the similarity to recurrent browsing contexts. **FM** [24] is a factorization machine-based recommendation method that considers the second-order interactions among feature fields. **DeepFM** [14] combines an FM component and a neural network component together to produce the final prediction. **PNN** [23] is a product-based neural network including embedding and product components. **MMoE** [22] is a neural network based on multi-gate mixture-of-experts model. **DIN** [39] uses a local activation unit to model the categorical features. **LSTM** [16] is the long-short term memory network to model the historical records; **GRU** [15] uses the gate recurrent units to model historical data. **RGCN** [26] combines graph convolutional networks with factorization models. **RGAT** [4] incorporates graph attention networks with factorization models. We apply all the baseline methods except RGCN and RGAT on the browsing logs of users (i.e., $\mathcal{H}$), and employ RGCN and RGAT on our relational graph (i.e., $\mathcal{G}$).

As described in Algorithm 1, there are two versions of MAGUS, each with a different approach to calculating the edge weights of the relational graph in line 2. We summarize them as follows. **MAGUS** denotes our MAGUS system, where the edge weights are computed using Eq. (7). **MAGUS⁺** is a variant of the MAGUS system, where the edge weights are calculated using Eq. (12). We use MAGUS+X and MAGUS⁺+X to denote using X as $\psi_{RE}(\cdot)$ in Algorithm 1.

In our experiment, we use 9 popular baseline methods, i.e., FM, DeepFM, PNN, MMoE, DIN, LSTM, GRU, RGCN, and RGAT, as the recommender system (i.e., $\psi_{RE}(\cdot)$) to form MAGUS. We then select four typical baseline methods, namely FM, DIN, GRU, and RGAT, as the recommender system to form MAGUS⁺.

Extension from Single-round setting to Multiple-round setting. As the proposed multiple-round recommendations for both queries and items is a new task, there is no existing approach particularly designed for the task. There are two main adjustments to adapt the above baseline methods to our setting. The first adjustment is to align their action space with the MAGUS system. Concretely, the action space needs to be extended from the item set (i.e., $\mathcal{V}$) to include both items and queries (i.e., $\mathcal{V} \cup \mathcal{Q}$). The other one is to enable them to utilize the user response during multiple-round interactions. To this end, we introduce a recurrent neural network, as illustrated in Figure 4. We first feed the last embedding layer of a given single-round baseline method into a unit, which allows passing the information of each round into the following rounds. Then, we, inspired by the label trick proposed in [18], establish a representation vector to represent every user behavior (e.g., clicks). And, we can represent each user response and feed it into the unit.

In our implementation, we build and learn two representation vectors to represent positive and negative responses from users respectively.

Implementation Details. The learning rate is decreased from the initial value 1×10^{-2} to 1×10^{-5} during the training process. The dimensions of all the embedding vectors including users and words (and queries and items) are set as 64. The batch size is set as 1000. The weight for L2 regularization term is 4×10^{-4} . The dropout rate is set as 0.5. We set the length of recommendation lists (denoted as

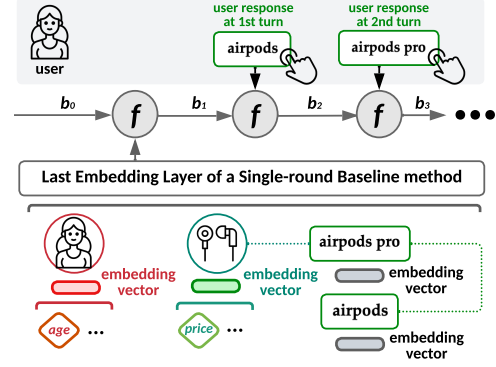


Figure 4: An illustrated example of extending a single-round baseline method to our multiple-round setting, where we design a recurrent neural network to pass the information of each round to the following rounds.

B) as 3. The maximum number of rounds (denoted as K_{MAX}) is set in the evaluation metrics (see Section 5.3 for details).

5.3 Evaluation Metrics

Note that as introduced in *Definition 2.1*, the objective of the MAGUS system is to identify an item satisfying user needs in each session efficiently. For each experimental setting, we first set K_{MAX} , and then evaluate the performance in terms of two types of metrics.

One is round-wise accuracy (denoted as $RA@K_{MAX}$), evaluating the recommendation performance. At each round, suppose a user clicks on the recommendation at position b_k at each k round, then we set $RA@K_{MAX} = 1/\log_2(b_k + 1)$; and if a user clicks on no recommendations, then $RA@K_{MAX} = 0$. We report the final $RA@K_{MAX}$ as the average of all the $RA@K_{MAX}$ s.

The other one is session-wise accuracy (denoted as $SA@K_{MAX}$), evaluating whether the system successfully recommends a target item within K_{MAX} interactions in each session. Concretely, we set $SA@K_{MAX} = 1$ when the system succeeds, and set $SA@K_{MAX} = 0$ when the system fails. We report the final $SA@K_{MAX}$ as the average of all the $SA@K_{MAX}$ s.

To verify the benefits of combining query recommendations and item recommendations, we also report the single-round accuracy (denoted as SAC) of each baseline method. Specifically, in each session, if the recommendation list includes at least one target item, then $SAC=1$; otherwise, $SAC=0$. We report the final SAC as the average of all the SACs.

5.4 Performance Evaluation

We report our experimental results in Table 1. We summarize our findings as follows.

Joint Recommendations helps (single) Item Recommendations except for Popularity-based Methods. By comparing the methods in terms of SAC, SR@3, and SR@5, we can see that joint recommendations can consistently help item recommendations with 11 different recommender bases, including tabular recommendation methods (e.g., FM), sequential recommendation methods (e.g., GRU), and graph-based methods (e.g., RGAT), on 3 different

Table 1: Results comparison of items recommendations in terms of SAC, and joint recommendations of both queries and items in terms of RA@3, SA@3, and SA@5. Since SAC metric measures the performance on the single-round item recommendation task, we do not report SAC for MAGUS and MAGUS⁺.

Methods	Amazon				Alipay				Tmall			
	SAC	RA@3	SA@3	SA@5	SAC	RA@3	SA@3	SA@5	SAC	RA@3	SA@3	SA@5
MPS	0.332	0.612	0.174	0.255	0.298	0.541	0.125	0.181	0.312	0.575	0.164	0.208
Hybrid	0.394	0.665	0.312	0.406	0.365	0.592	0.286	0.345	0.344	0.592	0.295	0.337
FM	0.634	0.773	0.672	0.757	0.716	0.815	0.767	0.846	0.718	0.832	0.745	0.824
FM+MAGUS	/	0.816	0.742	0.798	/	0.843	0.825	0.888	/	0.894	0.791	0.877
FM+MAGUS ⁺	/	0.825	0.753	0.802	/	0.848	0.832	0.890	/	0.895	0.795	0.878
DeepFM	0.676	0.784	0.693	0.798	0.730	0.825	0.787	0.875	0.729	0.843	0.766	0.841
DeepFM+MAGUS	/	0.833	0.767	0.811	/	0.851	0.832	0.895	/	0.903	0.814	0.892
PNN	0.688	0.788	0.690	0.792	0.741	0.833	0.775	0.870	0.722	0.823	0.753	0.831
PNN+MAGUS	/	0.839	0.772	0.817	/	0.865	0.852	0.911	/	0.884	0.812	0.876
MMoE	0.631	0.770	0.663	0.744	0.703	0.802	0.745	0.811	0.723	0.842	0.752	0.830
MMoE+MAGUS	/	0.801	0.725	0.776	/	0.833	0.820	0.876	/	0.898	0.802	0.881
DIN	0.697	0.798	0.696	0.813	0.757	0.845	0.793	0.886	0.736	0.855	0.774	0.848
DIN+MAGUS	/	0.845	0.775	0.828	/	0.878	0.865	0.922	/	0.904	0.818	0.902
DIN+MAGUS ⁺	/	0.849	0.777	0.830	/	0.881	0.872	0.924	/	0.908	0.820	0.908
LSTM	0.692	0.789	0.690	0.808	0.752	0.840	0.782	0.876	0.728	0.846	0.759	0.837
LSTM+MAGUS	/	0.840	0.773	0.821	/	0.870	0.861	0.918	/	0.901	0.808	0.892
GRU	0.707	0.803	0.699	0.818	0.762	0.848	0.799	0.889	0.732	0.852	0.771	0.845
GRU+MAGUS	/	0.848	0.788	0.831	/	0.882	0.871	0.926	/	0.909	0.821	0.901
GRU+MAGUS ⁺	/	0.850	0.783	0.828	/	0.884	0.875	0.924	/	0.912	0.824	0.906
RGCN	0.668	0.781	0.687	0.784	0.736	0.828	0.785	0.877	0.722	0.828	0.747	0.825
RGCN+MAGUS	/	0.841	0.775	0.824	/	0.873	0.860	0.912	/	0.897	0.810	0.893
RGAT	0.675	0.785	0.695	0.794	0.748	0.838	0.782	0.878	0.730	0.843	0.762	0.839
RGAT+MAGUS	/	0.850	0.785	0.828	/	0.878	0.868	0.921	/	0.905	0.817	0.897
RGAT+MAGUS ⁺	/	0.852	0.787	0.831	/	0.882	0.870	0.925	/	0.903	0.820	0.902

datasets. One possible explanation is that the multiple-round setting, compared to the single-round setting, would get real-time user preference, and thus the recommender system built upon the multiple-round setting can better capture user needs. We note that the popularity-based methods (i.e., MPS, Hybrid) perform well in terms of RA@3 but perform poorly in terms of SA@3 and SA@5. The main reason would be that high popularity mainly occurs in queries and low popularity often happens in items, which leads the recommender system to recommend queries too often.

Graph-based methods work well on Joint Recommendations.

By comparing the baseline methods in terms of SAC, we can see that the sequential recommendation methods (i.e., GRU, LSTM, DIN) can outperform other baseline methods in most cases. However, when comparing the baseline methods in terms of RA@3, SA@3, and SA@5, the graph-based methods (i.e., RGAT and RGCN) can

achieve comparable or better performance than the sequential recommendation methods. One possible reason is that graph-based methods working on our relational graph can effectively leverage the dependence among queries and items to make recommendations. This evidence also helps to verify the superiority of using a relational graph to organize queries and items.

Our MAGUS can consistently outperform Recursive Framework depicted in Figure 4. By comparing X+MAGUS and X in terms of RA@3, SA@3, and SA@5 (where X is one of 9 baseline methods), we can see that X+MAGUS can significantly outperform X, which indicates that MAGUS provides a better solution of bridging query recommendations and item recommendations. One potential reason for the improved performance of MAGUS could be its ability to leverage the dependence among queries and items

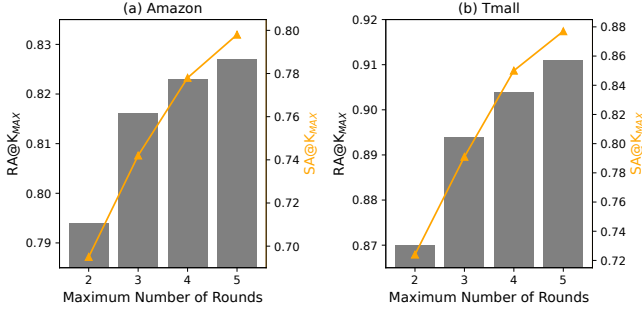


Figure 5: Performance comparisons of MAGUS with different maximum numbers of rounds (i.e., K_{MAX}) in terms of $RA@K_{MAX}$ and $SA@K_{MAX}$ with the FM recommendation base.

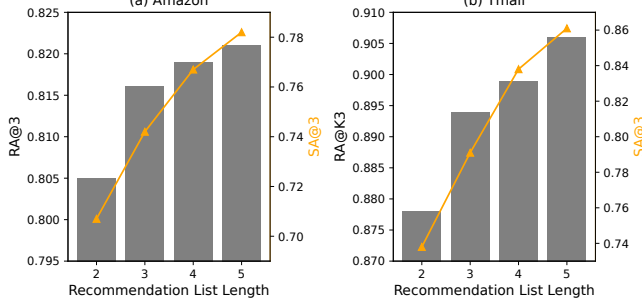


Figure 6: Performance comparisons of MAGUS using different lengths of the recommendation list in terms of $RA@3$ and $SA@3$ with the FM recommendation base.

through the label propagation algorithm. This allows the recommender system to encode the user’s preferences and feedback over time, leading to more accurate and personalized recommendations.

Tuned Edge Weights are helpful. Comparisons between $X+MAGUS$ and $X+MAGUS^+$ (where X is FM or GRU or RGAT), we can see that $MAGUS^+$ suggests that the learned edge weights in $MAGUS^+$ are able to capture meaningful relationships between queries and items. Although the improvements might not be substantial, the consistent improvement indicates that incorporating these learned edge weights into the multiple-round recommendation process can have a positive impact.

5.5 Hyperparameter Study

Impact of Maximum Number of Rounds. A key hyper-parameter of MAGUS is the maximum number of rounds, denoted as K_{MAX} . We investigate the performance change of MAGUS by assigning $K = 2, 3, 4, 5$ on Amazon and Tmall datasets. Results depicted in Figure 5 show that the multiple-round recommendations with more round feedback from users consistently outperform those with less feedback. It demonstrates the advantage of developing a multiple-round recommendation service.

Impact of Recommendation Length. The length of the recommendation list at each round (denoted as B) is another key hyper-parameter in MAGUS. We investigate its impact by assigning $L = 2, 3, 4, 5$ on Amazon and Tmall datasets. Results shown in Figure 6 indicate that enlarging the length of the recommendation list can lead to improved performance. This improvement can be

attributed to the fact that a larger list provides users with more choices and options to choose from. However, its performance gain is not as significant as increasing the rounds, because compared to B recommendations in one round, recursively recommending 1 in B rounds, enables the model to update the predicted scores to benefit the following recommendations.

5.6 Complexity Study

We investigate the time complexity of MAGUS and $MAGUS^+$ based on the FM, GRU, and DIN recommendation bases on Tmall dataset. We report the training and inference times for one round of the whole data in Figure 7. From the figure, we can see that MAGUS almost makes no difference in the training phase, because the computations brought by MAGUS is building the relational graph, while $MAGUS^+$ would cause some computation costs as it needs to run the one-round feature propagation on the graph. For the inference phase, the computation costs of applying either MAGUS or $MAGUS^+$ are from the label propagation algorithm.

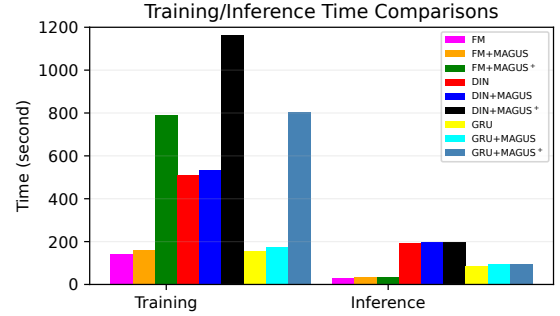


Figure 7: Comparisons of training and inference time of MAGUS and $MAGUS^+$ against several baselines on Tmall dataset.

5.7 Case Study

To better demonstrate the superiority of MAGUS, we conduct a case study on Amazon dataset. Figure 8 illustrates the process of MAGUS. MAGUS first initializes scores for nodes in the relational graph, where these nodes represent individual words and combinations of words served as queries and items. Then, after receiving the responses from the given user, MAGUS updates the scores accordingly. This dynamic process allows MAGUS to adapt to online user preferences and provide more personalized recommendations. We argue that it is quite meaningful in practice, because one simple and easy-to-deploy solution [2] is to always recommend a query or an item with high popularity to the user (i.e., MPS in the experiment part). For clarity, we also provide the level of popularity of each node in the figure. In this case, MAGUS recommends query clothes in the first round and item 1940 in the second round, while the popularity based method recommends query food in the first round and clothes in the second round. The comparison highlights that the popularity-based method overlooks the differences among users and the evolving nature of user preferences across different rounds. This limitation is similar to the popularity bias observed in item recommendation tasks, where popular items tend to dominate recommendations without considering individual user preferences.

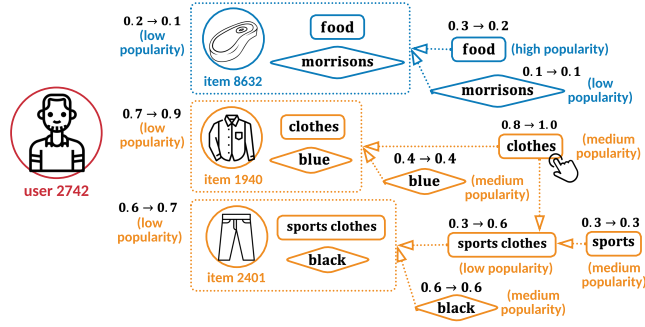


Figure 8: An illustrated case of MAGUS providing one recommendation to user 2742 at each round. The number depicted over each node is its score regarding the user. The popularity level of each node is computed according to its frequency in the user's browsing log.

6 CONCLUSION AND FUTURE WORK

In this paper, we propose a novel recommendation service of jointly recommending queries and items, and introduce a corresponding framework called MAGUS. In the MAGUS system, we build a relational graph and develop the corresponding label propagation algorithm, which can capture the dependence between queries and items. Our MAGUS can be seamlessly employed on any recommendation platform. In the future, it would be interesting to evaluate MAGUS on an online e-commerce platform.

REFERENCES

- [1] Wasi Uddin Ahmad, Kai-Wei Chang, and Hongning Wang. 2019. Context attentive document ranking and query suggestion. In *SIGIR*.
- [2] Ziv Bar-Yossef and Naama Kraus. 2011. Context-sensitive query auto-completion. In *WWW*.
- [3] Keping Bi, Qingyao Ai, Yongfeng Zhang, and W Bruce Croft. 2019. Conversational product search based on negative feedback. In *Proceedings of the 28th acm international conference on information and knowledge management*. 359–368.
- [4] Dan Busbridge, Dane Sherburn, Pietro Cavallo, and Nils Y Hammerla. 2019. Relational graph attention networks. *arXiv preprint arXiv:1904.05811* (2019).
- [5] Fei Cai, Maarten De Rijke, et al. 2016. A survey of query auto completion in information retrieval. *Foundations and Trends in Information Retrieval* (2016).
- [6] Ruey-Cheng Chen and Chia-Jung Lee. 2020. Incorporating behavioral hypotheses for query generation. *arXiv preprint arXiv:2010.02667* (2020).
- [7] Wanyu Chen, Fei Cai, Honghui Chen, and Maarten de Rijke. 2018. Attention-based hierarchical neural query suggestion. In *SIGIR*.
- [8] Konstantina Christakopoulou, Alex Beutel, Rui Li, Sagar Jain, and Ed H Chi. 2018. Q&R: A two-stage approach toward interactive recommendation. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 139–148.
- [9] Konstantina Christakopoulou, Filip Radlinski, and Katja Hofmann. 2016. Towards conversational recommender systems. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 815–824.
- [10] Mostafa Dehghani, Sascha Rothe, Enrique Alfonseca, and Pascal Fleury. 2017. Learning to attend, copy, and generate for session-based query suggestion. In *CIKM*.
- [11] Chongming Gao, Wenqiang Lei, Xiangnan He, Maarten de Rijke, and Tat-Seng Chua. 2021. Advances and challenges in conversational recommender systems: A survey. *AI Open* 2 (2021), 100–126.
- [12] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *ICML*.
- [13] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 249–256.
- [14] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *IJCAI* (2017).
- [15] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *ICLR* (2015).
- [16] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* (1997).
- [17] Dietmar Jannach, Ahtsham Manzoor, Wanling Cai, and Li Chen. 2021. A survey on conversational recommender systems. *ACM Computing Surveys (CSUR)* 54, 5 (2021), 1–36.
- [18] Jiarui Jin, Xianyu Chen, Weinan Zhang, Junjie Huang, Ziming Feng, and Yong Yu. 2022. Learn over Past, Evolve for Future: Search-based Time-aware Recommendation with Sequential Behavior Data. In *WWW*.
- [19] Wenqiang Lei, Xiangnan He, Yisong Miao, Qingyun Wu, Richang Hong, Min-Yen Kan, and Tat-Seng Chua. 2020. Estimation-action-reflection: Towards deep interaction between conversational and recommender systems. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 304–312.
- [20] Wenqiang Lei, Gangyi Zhang, Xiangnan He, Yisong Miao, Xiang Wang, Liang Chen, and Tat-Seng Chua. 2020. Interactive path reasoning on graph for conversational recommendation. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2073–2083.
- [21] Raymond Li, Samira Ebrahimi Kahou, Hannes Schulz, Vincent Michalski, Laurent Charlin, and Chris Pal. 2018. Towards deep conversational recommendations. *Advances in neural information processing systems* 31 (2018).
- [22] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H Chi. 2018. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *KDD*.
- [23] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-based neural networks for user response prediction. In *ICDM*.
- [24] Steffen Rendle. 2010. Factorization machines. In *ICDM*.
- [25] Paul Resnick and Hal R Varian. 1997. Recommender systems. *Commun. ACM* 40, 3 (1997), 56–58.
- [26] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *European semantic web conference*.
- [27] Alessandro Sordani, Yoshua Bengio, Hossein Vahabi, Christina Lioma, Jakob Grue Simonsen, and Jian-Yun Nie. 2015. A hierarchical recurrent encoder-decoder for generative context-aware query suggestion. In *CIKM*.
- [28] Yueming Sun and Yi Zhang. 2018. Conversational recommender system. In *SIGIR*.
- [29] Saeed Tahery and Saeed Farzi. 2020. Customized query auto-completion and suggestion—A review. *Information Systems* (2020).
- [30] Sida Wang, Weiwei Guo, Huiji Gao, and Bo Long. 2020. Efficient neural query auto completion. In *CIKM*.
- [31] Xiao Wang, Deyu Bo, Chuan Shi, Shaohua Fan, Yanfang Ye, and S Yu Philip. 2022. A survey on heterogeneous graph embedding: methods, techniques, applications and sources. *IEEE Transactions on Big Data* (2022).
- [32] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *SIGIR*.
- [33] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *WWW*.
- [34] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
- [35] Tong Yu, Yilin Shen, and Hongxia Jin. 2019. A visual dialog augmented interactive recommender system. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 157–165.
- [36] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *KDD*.
- [37] Xiaoying Zhang, Hong Xie, Hang Li, and John CS Lui. 2020. Conversational contextual bandit: Algorithm and application. In *Proceedings of the web conference 2020*. 662–672.
- [38] Yongfeng Zhang, Xu Chen, Qingyao Ai, Liu Yang, and W Bruce Croft. 2018. Towards conversational search and recommendation: System ask, user respond. In *Proceedings of the 27th acm international conference on information and knowledge management*. 177–186.
- [39] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *KDD*.